

CarbonRoute

A Reproducible Benchmark and Deadline-Risk Calibrator for Carbon-Aware Batch Scheduling under Forecast Error

Submitted by: Yuvika Nagpal (1024030141)

Batch: 3C15

Kumkum Gupta (1024030144)

Aaneya Sabharwal (1024030147)

Course: Software Engineering Date: 6 August 2026

1. Project Understanding & Motivation

Cloud computing workloads such as machine-learning training, backups, data analysis, video rendering, and software testing are often **flexible batch jobs**: they do not always need to execute immediately and may be scheduled at different times or across different regions.

The carbon intensity of electricity varies with both **time and location**. Therefore, a scheduler can potentially reduce emissions by selecting a cleaner execution time or region. However, this decision is made using forecasts, and forecasts are not always correct. In practice, a scheduling decision can also be affected by changing cloud prices, limited computing capacity, unexpected regional failures, workload surges, and job deadlines.

CarbonRoute will provide a controlled simulation and evaluation environment to study this problem. Instead of controlling real cloud infrastructure, the system will create reproducible cloud and workload scenarios and compare different scheduling policies under progressively more uncertain conditions.

The project does **not** claim to invent carbon-aware scheduling. Existing systems already support forecast-based carbon-aware placement. The proposed contribution is a **reproducible benchmark and deadline-risk calibration framework** for evaluating how reliable different scheduling strategies remain when their forecasts are imperfect.

2. Research Question

How accurately can deadline-violation risk be calibrated, and how effectively can an uncertainty-aware scheduler reduce carbon emissions while satisfying deadlines, resource constraints, and cost limits under imperfect carbon forecasts?

The study will investigate not only which policy achieves lower emissions, but also **when that policy becomes unreliable and how accurately it can estimate the risk of missing a deadline**.

3. Proposed System

CarbonRoute will consist of six integrated components:

A. Workload Generator

Generates configurable batch jobs containing:

- Release time and duration
- CPU and memory requirements
- Deadline and priority
- Maximum cost
- Permitted regions

B. Multi-Region Cloud Simulator

Models multiple simulated regions with:

- Computing capacity
- Memory availability
- Electricity carbon intensity
- Regional cost
- Running workloads
- Temporary capacity failures

C. Versioned Trace Builder

Maintains separate **forecast and realized carbon traces**, along with regional cost, capacity, and workload information. Each experiment will record its dataset version, random seed, workload parameters, algorithm configuration, and forecast-error setting so that experiments can be reproduced.

D. Scheduling Engine

The system will implement and compare:

1. Immediate execution

2. Earliest-deadline-first
3. Cost-aware scheduling
4. Deterministic carbon-aware scheduling
5. CarbonAware Scheduler baseline, where feasible
6. Proposed uncertainty-aware scheduler
7. Oracle scheduler as a best-possible reference

E. Stress-Test Engine

The scheduler will be evaluated under:

- Systematic forecast bias
- Random forecast error
- Missing forecasts
- Sudden regional capacity loss
- Dynamic price changes
- Workload surges
- Tight deadlines

F. Experiment Dashboard & API

A FastAPI-based interface and web dashboard will allow users to select workloads, algorithms, and uncertainty conditions, execute experiments, and compare their results through tables and visualizations.

4. Proposed Uncertainty-Aware Scheduler

The central contribution of CarbonRoute is a scheduler that does not blindly trust the predicted carbon intensity.

For every scheduling option, it will consider:

Expected carbon + cost + delay + deadline risk + forecast uncertainty

For example, consider a job that can either start immediately or wait two hours for a predicted cleaner period.

If the forecast is reliable and the deadline remains comfortably achievable, delaying the job may be beneficial.

If the forecast is highly uncertain and waiting creates significant deadline risk, the scheduler may choose to execute earlier even though the predicted carbon intensity is higher.

A scheduling decision will therefore produce an interpretable result such as:

Selected action: Start now

Expected carbon reduction: 18%

Forecast confidence: Low

Estimated deadline risk: 4%

The system will evaluate whether these predicted risks are actually reliable.

5. Risk Calibration

Suppose the scheduler predicts a **20% probability of deadline violation** for a group of similar jobs.

If approximately 20 out of 100 jobs actually miss their deadlines, the prediction is well calibrated.

If only 2 jobs miss their deadlines, the scheduler is overly pessimistic; if 40 miss, it is overly optimistic.

CarbonRoute will therefore evaluate **risk-calibration error** alongside carbon, cost, and deadline performance. This makes the project more than a simple carbon-minimization system: it studies whether the scheduler can **make decisions while correctly estimating the uncertainty associated with those decisions**.

6. Mathematical Objective & Constraints

The scheduling engine will optimize a weighted objective:

$$\text{Objective} = \alpha(\text{Carbon}) + \beta(\text{Cost}) + \gamma(\text{Deadline Penalty}) + \delta(\text{Delay}) + \lambda(\text{Forecast Risk})$$

where the weights determine the relative importance of each factor and will be documented and evaluated through sensitivity analysis.

Key constraints include:

- A job cannot start before its release time.
- Required CPU and memory must be available.
- The job should finish before its deadline.
- The job must remain within its cost limit.

- A scheduling decision must respect regional availability.

The **Oracle scheduler** will use realized future carbon values only as a reference for estimating the best possible outcome. It will not represent a deployable real-world scheduler.

7. Evaluation Methodology & Metrics

Each scheduling policy will be tested across multiple workloads, forecast-error levels, random seeds, resource conditions, and deadline settings. Results will be reported using averages and confidence intervals rather than a single favorable demonstration.

Primary metrics will include:

- Total realized carbon emissions
- Carbon reduction percentage
- Cloud cost
- Mean job delay
- Deadline-violation rate
- Risk-calibration error
- Regret relative to the Oracle
- Scheduler runtime
- Resource utilization
- Number of completed jobs

Regret will quantify how far a practical scheduler performs from the best possible decision under perfect knowledge.

8. Technical Architecture

Simulation & Data: Python, NumPy, Pandas, optionally SimPy

Scheduling & Optimization: Python algorithms/heuristics, optionally OR-Tools

Backend: FastAPI + PostgreSQL

Frontend: React + visualization library

Deployment: Docker and hosted dashboard/API

Development will begin with a small **single-job prototype** before expanding to multi-job, multi-region experiments. This reduces implementation risk and allows the scheduling logic to be validated incrementally.

9. Seventeen-Week Development Plan

Weeks Major Deliverable

- | | |
|-------|---|
| 1–2 | Requirements, research question, Python/simulation fundamentals |
| 3–4 | Job, region, resource and trace models |
| 5–6 | Immediate, deadline-first and cost-aware baselines |
| 7–8 | Carbon-aware scheduler and Oracle |
| 9–10 | Forecast-error generation and trace versioning |
| 11–12 | Uncertainty-aware scheduler and risk calibration |
| 13 | Capacity, price, workload and deadline stress tests |
| 14 | FastAPI experiment interface and dashboard |
| 15 | Repeated experiments and statistical evaluation |
| 16 | Testing, deployment, documentation and limitations |
| 17 | Final results, presentation and viva preparation |

10. Expected Outcome

The final system will provide a **reproducible benchmark, scheduling API, and experiment dashboard** for comparing carbon-aware scheduling policies under realistic uncertainty.

The project is expected to produce evidence answering questions such as:

- How much carbon reduction is actually achievable under forecast uncertainty?
- When does delaying a job become too risky?
- How sensitive are scheduling policies to forecast errors?

- Does the proposed uncertainty-aware scheduler reduce carbon without significantly increasing deadline violations?
- How close can practical scheduling policies get to the Oracle?

The final outcome will therefore be an **evidence-based evaluation of carbon-aware batch scheduling under uncertainty**, rather than simply another cloud dashboard or a basic carbon-minimization scheduler.